

# Final Report

**Title:** Visualizing Practical Byzantine Fault Tolerance (pBFT): An Interactive Consensus Simulator

**Authors:** A. Kumar, D. Lee, V. R. Osorio, X. Yan, Y. Zhang

**GitHub Repository:** <https://github.com/valeriar mire/720-pbft-visualization>

## Sections:

1. Introduction (Proof read required - rewrite)
2. Related Work (leave it as it is)
3. System Architecture
  1. Architectural Overview
  2. Deployment Topology
  3. Control Plane and Execution Lifecycle
  4. Log Ingestion and Event Transport
  5. Backend Event Consumption and Streaming
  6. Frontend Playback and Visualization
4. Backend Design & Implementation
  1. Design Goals and Constraints
  2. Control Plane and Execution Orchestration
  3. Log Generation and Redirection
  4. Event Transport via Redpanda
  5. Replica Reconfiguration API (done)
  6. Streaming layer API (done)
    - i. pBFT Visualization Data Model
    - ii. Request Buffering and Ordering Mechanism
5. Frontend Design & Implementation (React)
  1. Initial Design
  2. Technical High-Level Overview
  3. UI Composition & Controls
    - i. Top Panel
    - ii. Side Panel
    - iii. Main Canvas
  4. Hosting Frontend on the Webserver
6. Evaluation & Testing
  1. Functional Validation
    - i. Default case
    - ii. complex case
    - iii. edge case

2. Performance / Scalability
  - i.
7. Difficulties & Lessons Learned (EVERYONE CONTRIBUTE ATLEAST 1)
  1. Connectivity issues through github and ssh due to permissions and blocking.
  2. Playback logic: Implementing pause/step/back required a custom SSE buffer, snapshot history, and virtual time
  3. Dual Layout & Zoom: Supporting both ring and lane layouts on the same canvas meant duplicating drawing logic
  4. User-friendly UI design: Deciding on the optimal design for UI means trivial & complicated testing & refining.
  5. Complex permission issues with ssh into another ssh.
  6. Request grouping & buffering: Merging pending buffers when order–rank mappings arrive late, avoiding cross-round interleaving, and limiting buffer growth.
  7. Round boundary detection: Triggering flushes correctly on order/rank changes without premature or missed boundaries.
  8. Send control events on start and flush on exit
8. Limitations & Future Work (EVERYONE CONTRIBUTE ATLEAST 1)
  1. Only step-wise navigation is supported; there's no random access or "jump to event N" capability, and the history buffer holds roughly 80 steps before older snapshots are dropped.
  2. All state (layout, speed, faulty selection, history) lives in-memory; reloading the page wipes everything because we don't persist settings or events.
  3. Rendering is single-canvas and CPU-bound; large replica counts or high play speeds still risk frame drops, and there's no mobile/touch optimization.
  4. The event log is just a rolling list—no indexing, search, or filtering—so finding a specific event during long sessions remains manual.
  5. Visualization in each phase does animation one message by one message which does not reflect the real pbft behaviour and visualization for one round is significantly slowed (especially if we increase the number of replicas). Need to implement a new logic in both backend and frontend to fix this issue.
  6. Our current grouping and sorting strategy assumes that messages from the replicas arrive in the order of the requests they belong to. This assumption holds only when there is a single client issuing requests with sufficient spacing between them. If consecutive requests are sent too closely in time, the arrival order may interleave, and the backend can no longer rely on this ordering property.
9. Work Distribution (EVERYONE CONTRIBUTE ATLEAST 1)

See table below, just add it directly there

10. Conclusion (rewrite)

# 1. Introduction

This report outlines the Practical Byzantine Fault Tolerance (pBFT) consensus algorithm in a visualization tool composed on React, Python, and C++ programs. In general, pBFT shows the consensus agreement in a distributed network. Here, the number of nodes and replicas are monitored for their transaction and message information. In order to better visualize the process, this project proposes an user-friendly user interface (UI) to demonstrate the good and faulty replicas as well as the request, preprepare, prepare, commit, and inform stages to allow researchers to validate the actions. An interactive user interface for the pBFT visualization tool displays the stream of events in real-time (including user inputs) and allows users to modify the requests or change the number of replicas. The goal is to show the communication and transaction processing in a visual display. Currently, these logs are monitored using commands and with a large number of logs overtime, this can cause overhead for researchers and students. This tool was designed to help students and researchers visualize the behaviour of nodes and keep track of the messages, timestamps, and participants. It can be accessed by the public and the features of the user interface make it easy for any student to interact with the pBFT simulation. The rest of the report explains the motivation, user interface design, backend design, connectivity, and a discussion to explain our process and findings.

## 1.1 Objectives

The objectives of this project are:

- 1. Real-time visualization of pBFT executions**  
Provide a browser-based interface that animates pBFT protocol message flow and phase progression (*request, pre-prepare, prepare, commit, reply/inform*) using events produced by a running pBFT deployment.
- 2. Non-intrusive integration with an existing pBFT implementation**  
Integrate with provided pBFT C++ binaries without modifying the protocol implementation, by capturing structured JSON logs and transforming them into a visualization-friendly event stream.
- 3. Reliable event ingestion and protocol-aware reconstruction**  
Build a backend pipeline that aggregates logs from multiple distributed machines, tolerates out-of-order arrivals, and reconstructs logically ordered per-request event sequences without relying on synchronized clocks.
- 4. Interactive controls to support learning and experimentation**  
Provide UI features to start/reset runs, adjust the number of replicas, control playback speed and stepping, and export logs for replay or analysis.
- 5. Demonstrate fault-tolerance limits through crash-fault scenarios**  
Enable controlled replica termination during a run and visualize its effect on quorum formation and protocol progress, illustrating the boundary between successful commitment and stalled execution.

## 2. Related Work

There are only a few existing visualization tools for Practical Byzantine Fault Tolerance (pBFT) or similar protocols. One example is *ResView*, a web interface for the ResilientDB blockchain project that shows message flow, transaction rates, and pBFT progress among replicas (Exploratory Systems Lab, 2023). While useful, it is closely tied to ResilientDB's own system and not designed as a general teaching/learning tool.

Another example is an open-source project called *pBFT Visualization* (cmayurin, 2021), which animates pBFT message exchanges using a local simulation. However, it only demonstrates simplified pBFT logic and does not connect to a live backend or support variable replica counts.

Our project differs from these tools by focusing more on an educational visualization for a custom pBFT-style protocol currently under development. It would connect directly to a live running server that holds the schema, supports adjustable numbers of replicas, and provides a step-by-step view of nodes achieving consensus. These features make it more interactive and suitable for project demonstrations.

### 3. System Architecture

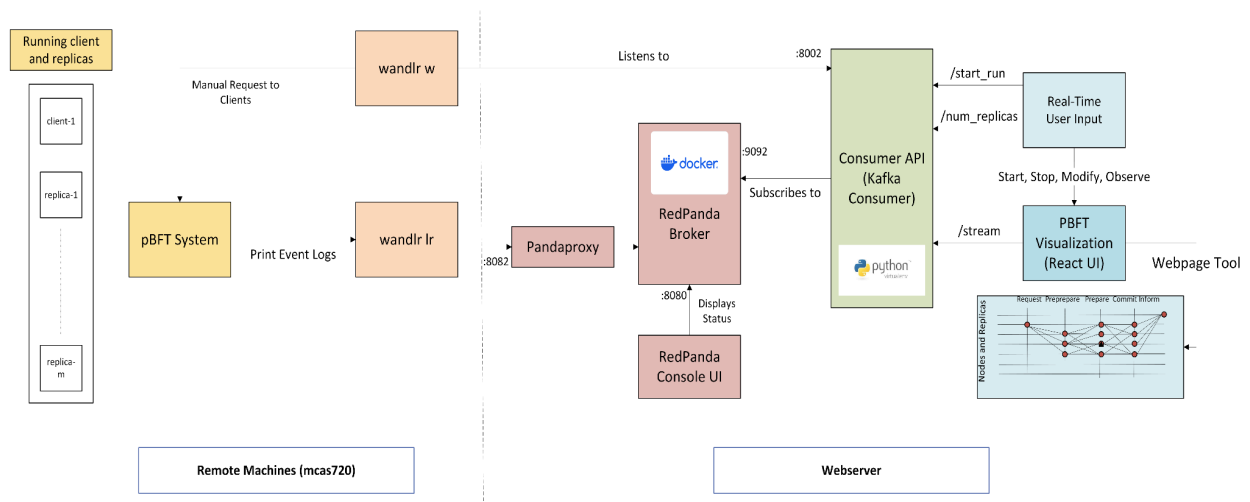
This section describes the overall architecture of the pBFT visualization system, focusing on how a real pBFT execution is orchestrated, logged, transported, and transformed into a form suitable for real-time visualization. The design emphasizes correctness of protocol semantics, reproducibility of executions, and robustness against partial failures during demonstration.

#### 3.1 Architectural Overview

The system adopts a centralized visualization architecture in which all orchestration, log ingestion, and event streaming are hosted on a dedicated webserver, while pBFT replicas and clients execute as independent processes on remote machines within the university network. This separation allows the visualization platform to observe and control a distributed pBFT execution without modifying the underlying protocol implementation.

At a high level, the architecture consists of four major components:

1. **pBFT Execution Layer:** A pre-existing pBFT implementation, provided as compiled C++ binaries, runs across multiple remote hosts. Each replica and client executes as an independent process and communicates using the pBFT protocol. The binaries are treated as black boxes and are not modified as part of this project.
2. **Log Ingestion & Event Transport Layer:** All pBFT protocol events are emitted as structured JSON log messages. These logs are intercepted and forwarded to a centralized, Kafka-compatible message broker (Redpanda), which acts as the durable event log for the entire system.
3. **Backend Consumer & Streaming API:** A backend service implemented using FastAPI consumes protocol events from Redpanda, reconstructs a logically ordered representation of pBFT rounds, and exposes the resulting event stream to the frontend via Server-Sent Events (SSE).
4. **Visualization Frontend:** A browser-based React application subscribes to the SSE stream and renders protocol progress in real time, displaying replicas, phases, and message flow animations.



### ***Figure 1: System Architecture Diagram***

This architecture (Figure 1) enables the visualization tool to observe real pBFT executions while remaining decoupled from both the protocol implementation and the frontend rendering logic.

## **3.2 Deployment Topology**

All backend services, including the message broker and the consumer API, are deployed on a single webserver. In contrast, pBFT replicas and clients are deployed on separate remote machines. Communication between components occurs over standard network interfaces and does not rely on shared memory or synchronized clocks.

The deployment model is summarized as follows:

- Remote Machines: Each pBFT replica and client runs on its own host. These processes communicate using the pBFT protocol and generate JSON-formatted log output describing received protocol messages.
- Webserver: Hosts the Redpanda broker (running in Docker), the FastAPI-based consumer service, and serves as the control plane for orchestrating pBFT executions via SSH.

This centralized backend design simplifies experiment management, log aggregation, and fault injection, while still preserving the distributed nature of the pBFT execution itself.

## **3.3 Deployment Topology**

The system provides a backend control plane that manages the lifecycle of pBFT executions. User actions in the web interface (e.g., starting a new run or changing the number of replicas) are translated into backend operations that perform the following tasks:

1. Terminate any existing pBFT processes on remote machines.
2. Regenerate configuration files based on the desired number of replicas.
3. Deploy updated binaries and configuration files to remote hosts via SSH.
4. Launch fresh pBFT replica and client processes.

This approach ensures that each visualization session corresponds to a clean execution of the pBFT protocol, avoiding interference from stale processes or residual state.

While centralized orchestration introduces a single point of control, it is well-suited to the educational and experimental goals of the project, where reproducibility and clarity of execution take precedence over deployment decentralization.

## 3.4 Log Ingestion and Event Transport

A key architectural challenge is capturing pBFT protocol activity from multiple distributed processes in a form suitable for visualization.

Each pBFT replica and client emits JSON log entries describing protocol events (e.g., request reception, prepare, commit). Rather than attempting to synchronize clocks across machines, the system treats logs as unordered event observations that must be interpreted using protocol semantics.

A custom tool, *wandlr*, is used in two modes:

- Log Redirect Mode (lr): Reads JSON log output from standard input and forwards each log entry to the Redpanda broker using Kafka's HTTP-compatible Pandaproxy interface.
- Workload Mode (w): Generates client requests at controlled intervals to drive pBFT execution.

All protocol events are published to a single Kafka topic with a single partition, ensuring a total arrival order at the broker. This ordering reflects ingestion order rather than true protocol order, which necessitates further processing at the consumer stage.

## 3.5 Backend Event Consumption and Streaming

The backend consumer service subscribes to the pBFT log topic and performs three primary functions:

1. Filtering and Normalization: Raw log entries are filtered to retain only protocol-relevant events and normalized into a common envelope format, capturing fields such as message type, sender, receiver, view number, and sequence identifiers.
2. Logical Ordering and Grouping: Because logs originate from different machines and may arrive out of order, the consumer reconstructs pBFT rounds using protocol-level identifiers (e.g., request rank and sequence/order numbers) rather than timestamps. Events belonging to the same logical request are buffered and later emitted in a deterministic, phase-respecting order.
3. Real-Time Streaming to the Frontend: Reconstructed events are streamed to the frontend using Server-Sent Events (SSE), enabling the visualization to update incrementally as the protocol progresses.

This design cleanly separates concerns: Redpanda provides durable event storage and decoupling, the consumer enforces protocol-aware ordering, and the frontend focuses solely on rendering.

## 3.6 Frontend Playback and Visualization

The React-based frontend subscribes to the SSE stream and maintains a local event buffer. A playback engine consumes buffered events at a speed controlled by the user (pause/step/fast-forward), dispatching them into a reducer that tracks replica phases, quorum progress, and faulty node markings. The renderer (a single HTML5 canvas) supports both ring and lane layouts, zoom, and hover inspection, highlighting



message paths and faulty replicas in real time. A snapshot stack enables step-back functionality, while reset/replica controls interact with the backend control plane to restart runs or adjust cluster size without reloading the page.

## 4. Backend Design & Implementation

This section describes the design and implementation of the backend infrastructure that orchestrates pBFT executions, captures protocol events, and prepares them for real-time visualization. The backend is responsible for managing experiment lifecycle, aggregating distributed logs, and enforcing protocol-aware ordering guarantees before events are streamed to the frontend.

### 4.1 Design Goals and Constraints

The backend was designed under several practical constraints that strongly influenced architectural decisions. Most importantly, the pBFT implementation used in this project was provided as a compiled binary and treated as a black box. As a result, the backend could not rely on internal protocol state, hooks, or instrumentation beyond the structured logs emitted by the binaries themselves.

Given this constraint, the backend design was guided by the following goals:

- The system must visualize protocol behavior as it occurs in a running pBFT deployment, rather than simulating or approximating protocol phases.
- pBFT replicas and clients execute on separate machines without synchronized timestamps. The backend must therefore reconstruct protocol progress using semantic identifiers (e.g., request ranks and sequence numbers), not wall-clock time.
- The backend must stream events incrementally for live visualization, while also enabling deterministic replay of completed executions.
- Each visualization session should correspond to a clean pBFT run with explicitly configured parameters, such as the number of replicas.
- The system prioritizes correctness, transparency, and debuggability rather than high-throughput or fault-tolerant backend deployment.

These goals motivate a centralized backend architecture with explicit orchestration and protocol-aware log processing.

### 4.2 Control Plane and Execution Orchestration

The backend implements a centralized control plane responsible for managing the full lifecycle of pBFT executions. All orchestration logic is hosted on the webserver and interacts with remote pBFT replicas and clients via SSH.

A visualization session begins when a user initiates a new run through the web interface. This action triggers backend control logic that performs the following steps:

1. Termination of existing processes: Any running pBFT replicas, clients, or auxiliary tools (e.g., wandlr) from a previous session are forcefully terminated on remote machines to prevent residual state from affecting the new execution.
2. Replica configuration selection: Based on the user-specified number of replicas, a filtered server configuration is generated by selecting the first  $N$  replica entries from a master server list (e.g., servers.data). This ensures consistent replica identity and ordering across runs.

3. Configuration regeneration: Fresh pBFT configuration files are generated for both replicas and clients using the provided configuration generator. These files encode network topology, cryptographic keys, and logging settings.
4. Remote deployment: The backend deploys the pBFT binaries, configuration files, and auxiliary tools to the appropriate remote hosts via SSH and SCP.
5. Process startup: Replica processes are launched first, followed by the client process, which generates one or more pBFT requests depending on the configured workload.

### 4.2.1 Centralized Control Rationale

Although centralized orchestration introduces a single point of control, it simplifies experiment management and reduces the likelihood of inconsistent system state. Given the educational focus of the project, this trade-off is acceptable and aligns with the goal of making pBFT behavior transparent and repeatable.

## 4.3 Log Generation and Redirection

A central challenge in visualizing pBFT is capturing protocol behavior from distributed processes without modifying the protocol implementation. This system addresses the challenge by relying exclusively on structured receive-side logging.

### 4.3.1 Receiver-Side Logging Model

Each pBFT replica and client emits JSON-formatted log entries describing protocol-relevant events, such as message reception, protocol phase transitions, and execution outcomes. Importantly, logs are generated only at the receiving process, not at the sender. This design choice ensures that logged events reflect observed protocol behavior rather than assumed message delivery.

While receive-side logging does not capture network-level events such as message loss or reordering directly, it provides a reliable and implementation-agnostic view of protocol progress from the perspective of each participant.

### 4.3.2 The `wandlr` Tool

To forward logs without altering the pBFT binaries, the system employs a custom auxiliary tool, wandlr, which operates in two modes:

- Log Redirect Mode: In this mode, wandlr reads JSON log lines from standard input and forwards each log entry to a centralized message broker via HTTP. The tool wraps each log entry with metadata identifying the receiving participant.
- Workload Mode: In workload mode, wandlr generates client-side requests at controlled intervals, driving the pBFT execution for a specified number of rounds.

By composing wandlr with standard input/output redirection, the backend can intercept pBFT logs transparently, without requiring access to the internal code of the protocol implementation.

## 4.4 Event Transport via Redpanda

All pBFT log events are transported through Redpanda, a Kafka-compatible streaming platform deployed on the webserver using Docker. Redpanda serves as the system's central, durable event log.

### 4.4.1 Kafka-Compatible Event Ingestion

pBFT log events are ingested using Kafka's HTTP-compatible Pandaproxy interface. Each log entry is published as a JSON record to a single topic dedicated to pBFT protocol events.

The system intentionally uses:

- A single topic `pbft-logs`
- A single partition

This configuration enforces a total arrival order at the broker, simplifying downstream consumption and replay. While this order does not reflect true protocol causality, it provides a deterministic ingestion sequence that can be reprocessed consistently.

### 4.4.2 Decoupling and Durability

Introducing a message broker between pBFT execution and visualization provides several advantages:

- **Decoupling:** pBFT replicas and clients operate independently of the visualization backend. Temporary backend outages do not disrupt protocol execution.
- **Durability:** All protocol events are persisted, enabling replay and post-hoc analysis.
- **Controlled consumption:** The backend consumer can process events at its own pace, buffer incomplete protocol rounds, and emit logically ordered events to the frontend.

By treating Redpanda as the authoritative event store, the backend cleanly separates event capture from protocol interpretation and visualization.

## 4.5 Replica Reconfiguration API

To support controlled pBFT experiments, the system provides a small set of management endpoints that govern replica configuration and execution lifecycle. These endpoints allow users to start a new protocol run, reset the system state, and change the number of replicas between experiments. Together, they define a simple and repeatable workflow for running pBFT under different configurations.

### 4.5.1 Replica Count Reconfiguration

The `/num_replicas` endpoint updates the number of pBFT replicas used in the system. When this endpoint is called, the backend terminates any running pBFT process, regenerates configuration files, and prepares the system for a new execution with the specified replica count. After a replica reconfiguration,

the Streaming API resets internal state related to buffering and control events to ensure that subsequent visualizations reflect a clean execution environment.

### 4.5.2 Starting a New pBFT Execution

The `/start_run` endpoint starts a new pBFT execution. It allows the user to set the request message and the number of rounds to run. When called, the system stops any existing pBFT processes and launches a fresh execution with the given parameters. Starting a new run also resets the internal execution state. The backend emits a new set of control events so that the frontend can clear the previous visualization and display the new run independently.

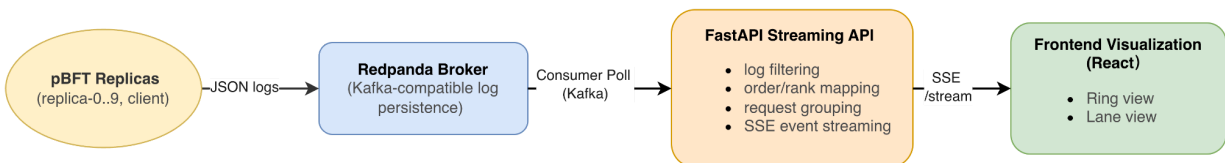
### 4.5.3 Resetting Execution State

The `/reset_run` endpoint stops any running pBFT execution without starting a new one. It clears the current request and buffered events maintained by the system. This endpoint is used to return the system to a clean idle state, for example after an interrupted run.

## 4.6 Streaming layer API

The Streaming layer API serves as the bridge between the Redpanda event log and the React-based visualization interface. Because pBFT replicas generate protocol messages continuously during each consensus round, the system requires a mechanism that can capture logs in real time, interpret their semantics, and deliver them to the frontend.

### 4.6.1 Architectural Overview



**Figure 2. End-to-end architecture of the pBFT log streaming and visualization system.**

Figure 2 illustrates the end-to-end data flow of our pBFT visualization system. pBFT replicas and clients emit JSON logs describing protocol messages, which are collected and persisted by a Kafka-compatible Redpanda broker. The Streaming API then consumes these logs, performs filtering, order/rank reconciliation, and request-level grouping, and streams the processed events to the frontend using SSE. The React-based frontend receives this live event stream and renders the protocol execution through Ring and Lane visualizations.

## 4.6.2 pBFT Visualization Data Model

The Streaming API converts raw pBFT logs into a clean and uniform format that can be replayed deterministically by the visualization frontend.

pBFT replicas and clients emit log entries in the form of `log_message_event`, a JSON-based log record produced by the C++ logging subsystem. The Streaming API first filters the input stream and keeps only known pBFT message types, including *ClientRequest*, *PrePrepare*, *Prepare*, *Commit*, and *Reply*.

For each retained log entry, the Streaming API extracts a small set of protocol-relevant fields. The sender of a message is read directly from the numeric participant identifier recorded in the log. The receiver appears as a string label such as *replica-i* or *client-j* and is converted into a numeric ID. Replica IDs use zero-based indices, while client IDs are assigned after all replica IDs so that all participants share a single ID space.

Request identity is not determined using client or request identifiers, since these fields do not appear consistently in all pBFT messages. Instead, the Streaming API reconstructs request identity from pBFT's ordering information. *ClientRequest* messages carry a client-side rank, *Prepare* and *Commit* messages carry the global order assigned by the primary, and *PrePrepare* messages link these two values together. Because rank and order appear reliably across the protocol and directly reflect how pBFT organizes requests, they are used to group messages belonging to the same consensus round.

After metadata normalization, the Streaming API converts each pBFT message into a unified event format that can be streamed directly to the frontend. This event format is independent of the original log structure and is designed specifically for visualization and replay.

| Streamable pBFT Event |                      |
|-----------------------|----------------------|
| type                  | Protocol phase       |
| view                  | View number          |
| seq                   | Request ordering key |
| from                  | Sender ID            |
| to                    | Receiver ID          |
| ts                    | Timestamp            |
| eid                   | Event ID             |
| data                  | Raw pBFT payload     |

Each event follows a fixed schema shared by all pBFT message types, summarized in Table 1. It includes the protocol phase, view number, request ordering key, sender and receiver identifiers, and a timestamp. A

monotonically increasing event identifier is assigned by the backend to preserve a total order over the streamed events and to support deterministic playback on the frontend.

The original pBFT message payload is preserved as part of the event data. This allows the frontend to access detailed protocol information when needed, while keeping the event structure stable and compact.

Events are serialized in a format compatible with SSE and streamed incrementally to the browser.

### 4.6.3 Request Buffering and Ordering Mechanism

In our pBFT visualization system, log messages generated by replicas do not arrive at the backend in the correct protocol order. This mismatch is unavoidable: events may be emitted by different threads within a replica, reordered by logging and transmission, and associated with unsynchronized system clocks across replicas. As a result, neither message arrival order nor timestamps reliably reflect the logical pBFT execution order. However, the visualization requires events to be presented in this logical sequence so that each pBFT round appears as a coherent and interpretable unit.

To handle out-of-order log messages, the backend maintains a buffer for each pBFT request. Instead of streaming messages immediately upon arrival, the backend temporarily stores them in per-request buffers. This allows the system to wait until sufficient information is available to determine each message's position in the final sequence, reconstructing the intended pBFT ordering through grouping and sorting before events are released to the frontend.

Each buffer stores three pieces of information:

- **The events themselves:** These are the raw message envelopes that will eventually be streamed to the frontend.
- **Metadata used for sorting:** For each event, the buffer records its phase rank (ClientRequest = 0, PrePrepare = 1, Prepare = 2, Commit = 3, Reply = 4). Inside each phase, the backend sorts events primarily by sender id, with receiver id used as a secondary key.
- **Timestamps of when the buffer was first and last updated:** These help detect idle buffers and trigger automatic flushing when a round has effectively ended.

While this information is sufficient to organize events once they are known to belong to the same request, identifying the request of an incoming message is not always possible, because different pBFT message types carry different pieces of identifying information.

Specifically:

- **ClientRequest** messages include only a *rank* value.
- **Prepare** and **Commit** messages include only an *order* value.
- **PrePrepare** and **Reply** messages include both *order* and *rank*, and therefore serve as the points where these two identifiers can be matched.

Because order and rank do not necessarily appear together early in the stream, the backend must temporarily treat them as partial, independent clues about which request is being executed. The process operates in three stages:

- **Handling messages with only rank:** ClientRequest messages are initially stored using their rank as the only known identifier. At this moment, the backend cannot yet associate them with a specific order.
- **Handling messages with only order:** Prepare and Commit messages are stored separately using their order value. Without a corresponding rank, the request identity is still incomplete.
- **Binding order and rank when both appear:** When a PrePrepare or Reply message arrives, it carries both identifiers. At this moment, the backend can finally determine which rank corresponds to which order. Once this relationship becomes known, all previously stored events that reference either the rank or the order can be merged into a single, definitive buffer representing that pBFT request.

As a result, before order–rank reconciliation, messages may be temporarily stored in separate “pending” buffers: one indexed by rank and another by order. After the backend discovers that a specific order corresponds to a specific rank, these temporary buffers are merged into a final request buffer. The merging process combines both the accumulated events and their associated metadata, preserving their timestamps and ensuring that no information is lost. From this point onward, all new events that reference either the order or the rank of this request are routed directly into the final buffer.

A buffer is not streamed to the frontend immediately after being created; instead, it is flushed only when the system determines that the request’s event sequence is complete or no further events are expected. Several conditions may trigger flushing:

- **Completion of a pBFT round:** When a new round begins (detected through a change in order or rank), the previous round’s buffer is finalized and flushed.
- **Idle timeout:** If a buffer receives no new events for a configurable period, the backend treats the round as complete and flushes the buffer to avoid indefinite waiting.

When a round is finalized, either due to the arrival of a new request or an idle timeout, the backend performs a final ordering of the events within the buffer, producing a consistent sequence that respects the pBFT phase progression. This ordered sequence is then streamed to the frontend as the complete representation of that round. After flushing, the backend clears its internal state for that request, allowing subsequent rounds to be processed independently.

Through these mechanisms, the backend can clearly identify the boundaries between consecutive pBFT rounds and deliver each round as a coherent, self-contained unit, even when log messages arrive late or out of order.



## 5. Frontend Design & Implementation

This section covers the browser-side architecture that consumes the streaming event feed, manages playback state, and renders PBFT progress in real time. The frontend’s role is to turn a protocol trace into an interactive learning experience, exposing enough control and context for users to explore both the mechanics and the intuition behind PBFT.

### 5.1 Design Principle

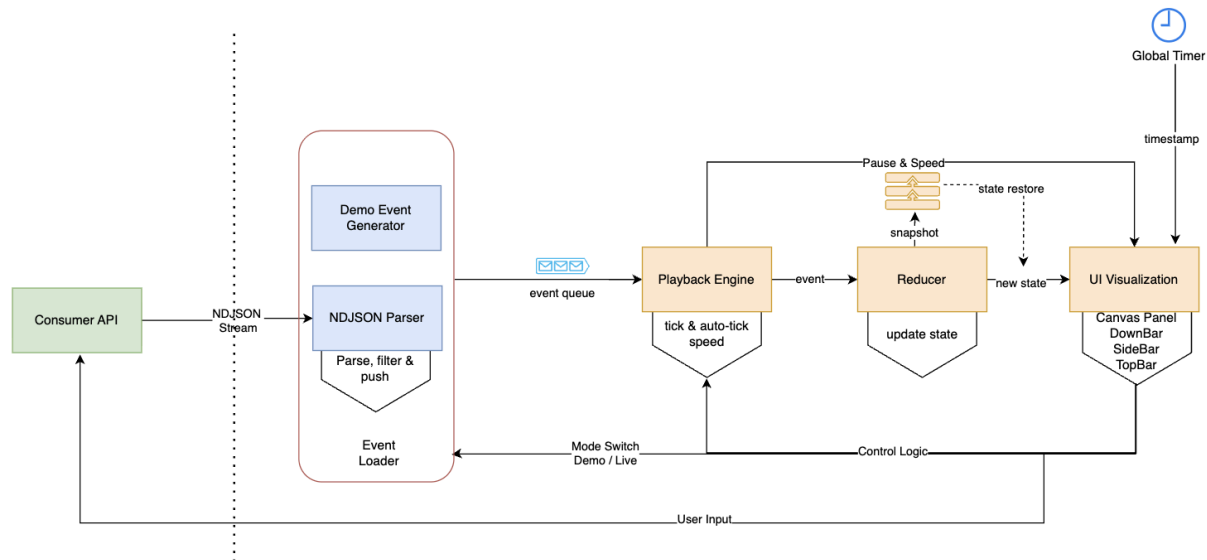
The frontend design was shaped by a few practical constraints. We never modify the PBFT binaries themselves, so the UI can only observe the event stream exposed by the backend; it must remain source-agnostic and tolerant of imperfect logs. At the same time, it has to be approachable for students while still useful for researchers who expect fine-grained control.

Given those constraints, we set the following goals:

- **Faithful playback, not simulation:** the UI should render exactly what the backend streamed, including real quorum counts and faulty sets, rather than animating a canned scenario.
- **Multiple mental models:** the renderer must support both “lanes” (phase timeline) and “ring” (communication graph) layouts so viewers can switch perspectives without losing context.
- **Offline + live parity:** the reader mode (demo) and the live SSE mode must feel identical aside from input source. Demo runs need manual/auto stepping for classrooms, while live runs consume the real stream; both share the same reducer, controls, and layouts so users can switch between “what-if” demos and real executions without relearning the UI.
- **Interactive pacing:** users need deterministic pause/resume, step/back, and speed control. That required buffering every SSE event locally and driving playback from a consistent clock.
- **Observable protocol state:** key metrics (n, f cap, faulty IDs, quorum progress, SSE stats) have to remain visible and synchronized with the canvas so users always know what the system believes.
- **Recoverable sessions:** reset/replica controls should clear buffers, histories, and faulty halos, then coordinate with the backend to start a fresh run—mirroring how experiments are orchestrated on the server.

These goals led us to pair a reducer-driven React app with a custom playback engine and a single canvas renderer, ensuring the frontend stays faithful to the protocol trace while remaining interactive and educational.

### 5.2 Technical High-Level Overview



**Figure 3. Frontend Technical High-Level Architecture**

The frontend sits downstream from the backend consumer API, which publishes a continuous NDJSON stream of protocol envelopes. An “event loader” layer accepts this stream (or, in demo mode, a synthetic event generator), parses each entry, and pushes it into a local queue. From that queue, a playback engine governs when events are consumed: pause/back/next controls, and a tick interval derived from the user-selected speed.

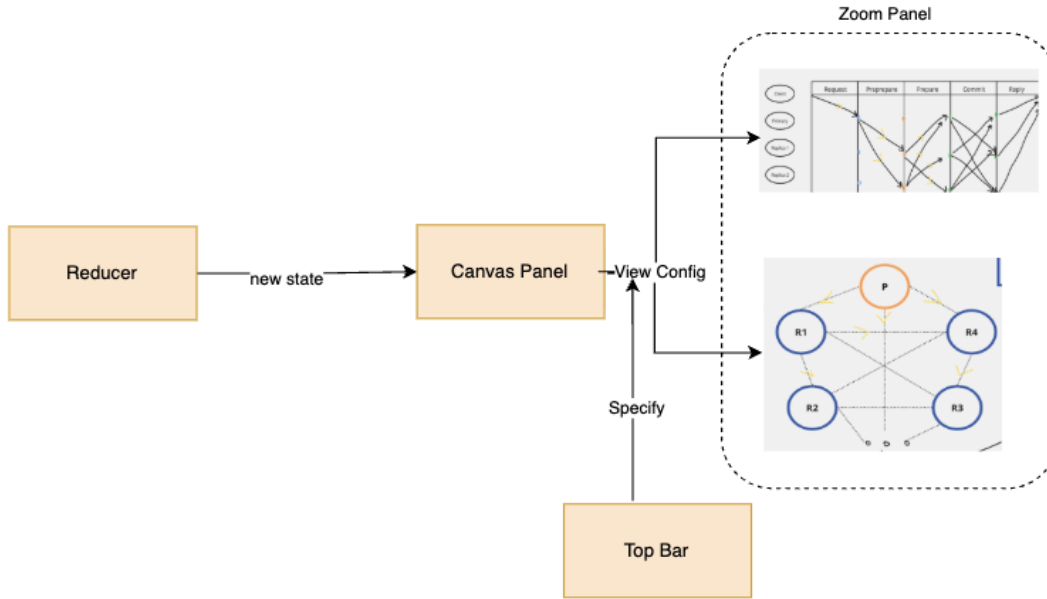
Each dequeued event is dispatched to a reducer that holds the canonical PBFT visualization state—replica phases, quorum counts, faulty IDs, log entries, etc. The reducer is deterministic and symmetrical across both modes; the only difference is where events originate. Whenever the user steps backward, the reducer restores a snapshot drawn from the history stack before resuming playback.

Finally, a global timer drives canvas animations while the UI layer (Canvas Panel, Sidebar, TopBar, DownBar) renders the latest reducer state. Because the visualization is fed by the same event queue regardless of source, demo and live runs remain feature-identical: only the loader changes, not the playback, state, or rendering logic.

## 5.3 Visualization and Controls

Section 5.3 dives into the moving pieces that turn an event stream into an interactive visualization: the canvas renderer, the timer-driven message animation model, and the playback controls (speed, pause, next/back) that keep the reducer, event queue, and UI in sync. The following subsections break down how these components work together—from drawing nodes to moving diamonds, throttling throughput, freezing time, and rewinding snapshots.

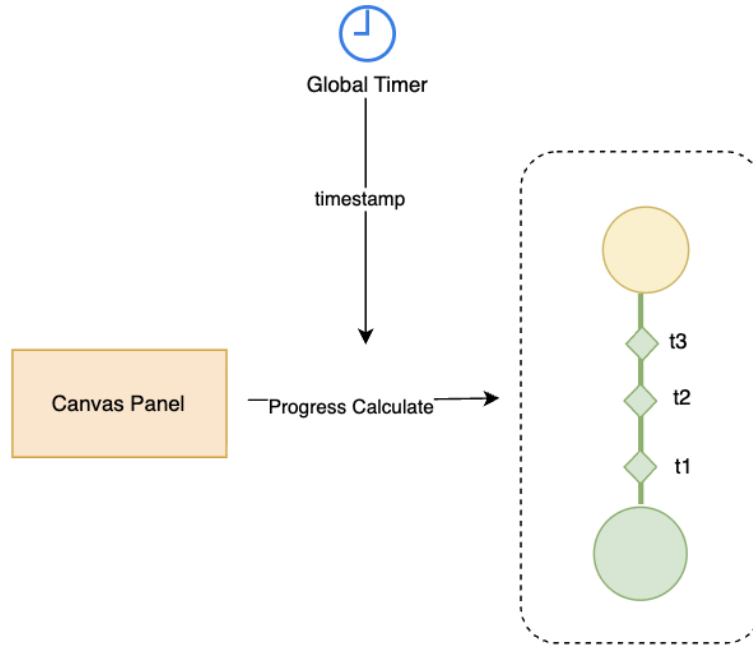
### 5.3.1 Basic Canvas Rendering Pipeline



**Figure 4. Basic Canvas Rendering Pipeline**

As indicated in 5.2, the reducer produces the authoritative PBFT state—replica phases, message pulses, quorum progress, faulty IDs. Each time this state changes, the `CanvasPanel` re-renders by passing the state and layout configuration into `useCanvasRenderer`. The `Top Bar` controls the view configuration (ring vs. lanes, zoom level), so toggling a layout button simply updates a React state that `CanvasPanel` reads on the next render. Because the renderer sits between the reducer and the canvas element, the same pipeline works for both layouts: the reducer emits a new state, `CanvasPanel` reads the current view settings, and the renderer draws the appropriate visualization. The zoom panel is just an overlay that scales the canvas via CSS transforms while the underlying state and drawing logic remain unchanged.

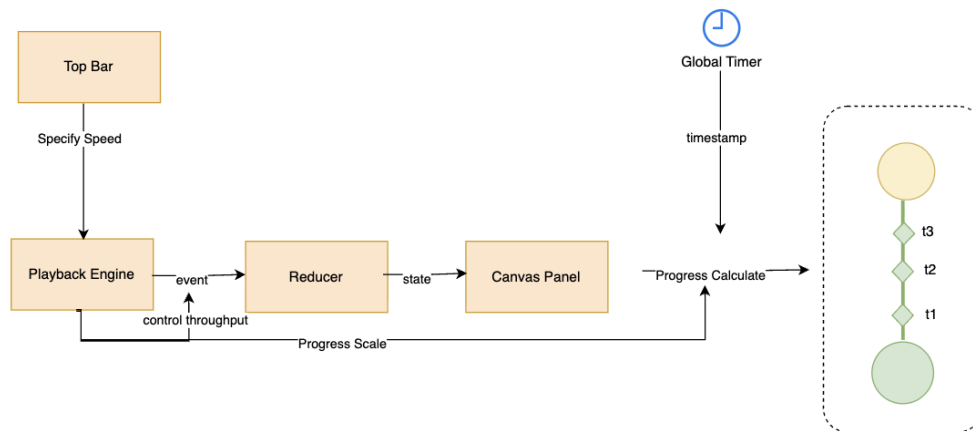
### 5.3.2 Message Animation Model (Timer-Driven Diamond Movement)



**Figure 5. Timer-Driven Diamond Movement**

To make the visualization more engaging and to help users track individual messages, we render each in-flight message as a “diamond” that travels along the edge between two replicas. Every time the reducer dispatches a new event, we record the current virtual time (`m.t = simTimeRef.current`). The canvas renderer runs in a `requestAnimationFrame` loop that reads the latest timestamp, computes `progress = (now - m.t) / flightMs`, clamps it to `[0,1]`, and interpolates the diamond’s position between source and destination coordinates. When progress reaches 1 the diamond disappears, leaving the node’s phase and quorum state updated.

### 5.3.3 Message Animation Model - Speed Control

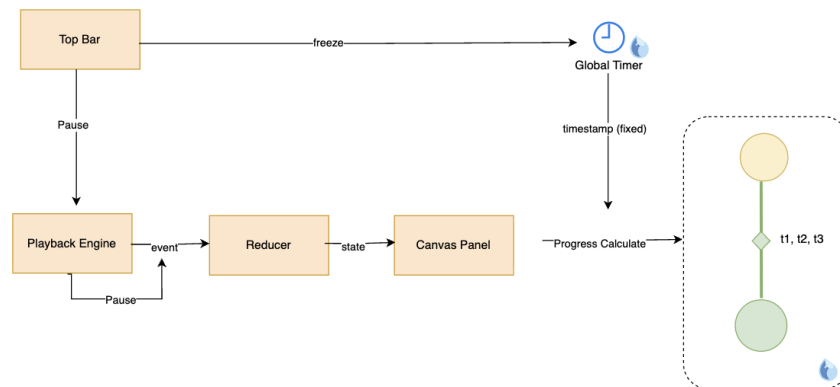


**Figure 5. Speed Control Logic**

The slider in the Top Bar drives a single “events per ten seconds” value (`demoEps`) that feeds the playback engine. We convert that into a tick interval (`tickMs = 10000 / demoEps`) and a flight duration (`flightMs ≈ 0.8 · tickMs`). Live mode uses `tickMs` as the cadence for dequeuing SSE events; demo mode uses the same interval for its scripted stages. When the user pauses, steps, or adjusts the slider, the playback engine immediately changes the throughput, so the reducer continues to receive the same event sequence but at a faster or slower rate.

Because every rendered diamond stores the timestamp captured when the reducer processed the event, changing `flightMs` also scales the animation speed. In effect, the playback engine controls how frequently events are dispatched, while the canvas’ progress calculation determines how quickly each diamond traverses its edge. This two-pronged approach keeps the visual timing consistent with the input stream, regardless of whether the source is live SSE or the offline demo driver.

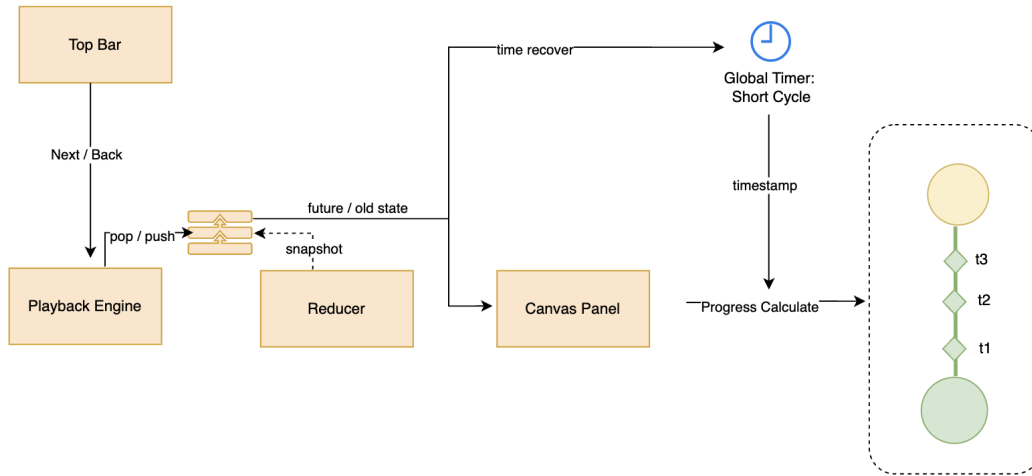
### 5.3.4 Message Animation Model - Pause



**Figure 6. Pause Logic**

When the user presses “Pause”, two things happen simultaneously. First, the playback engine stops dequeuing events from the buffer, so the reducer receives no new updates and the protocol state remains frozen. Second, the `requestAnimationFrame` loop that drives the canvas stops advancing the virtual timestamp unless there is a short “finish this animation” window scheduled. With the clock frozen, each message diamond keeps the same normalized progress (`progress = 0.35`, for example) and simply stays in place until playback resumes. This dual freeze ensures that pausing mid-flight leaves the visualization perfectly synchronized: no new actions are processed, and the existing animation frame does not drift. When the user hits “Continue”, event consumption restarts and the timer begins advancing again, so diamonds continue along their paths exactly where they left off.

### 5.3.5 Message Animation Model - Next / Back



**Figure 7. Next / Back Logic**

Back/Next are built on a snapshot stack. Before the reducer processes any event (live or demo), we capture a snapshot containing the reducer state, the playback cursor (`demoRef`), and the current virtual timestamp. Pressing “Back” pops the latest snapshot, restores the reducer state, and rewinds `simTimeRef` to the timestamp recorded in that snapshot. Immediately after restoring, we schedule a short animation window so the just-restored event replays once—even while paused—giving the user visual confirmation of the step they revisited. “Next” behaves symmetrically: if we have snapshots in the “future” stack, we pop one and restore it; otherwise we let the playback engine dequeue the next event from the queue and push a new snapshot. Because both controls operate on deterministic snapshots, they work the same in live and demo modes, and users can scrub back and forth without losing synchronization between the reducer state and the canvas animation.

#### 5.4 Side Panel

The side panel forms the primary informational and interpretive component of the pBFT Visualization Interface’s front end. While the canvas displays replica interactions and message exchanges visually, the side panel provides structured context, explanatory detail, and protocol-level insight. Its design integrates real-time system data, dynamic explanations, message-type indicators, and educational sections, allowing users to understand not only what the system is doing, but why it is doing it. The side panel is divided into several collapsible sections to support clarity, reduce cognitive load, and allow users to focus on elements most relevant to their current task.

At the top of the panel, the interface presents essential live pBFT system parameters in a compact key–value layout. These include the total number of replicas ( $n$ ), the Byzantine fault tolerance parameter ( $f$ ), the current protocol view, the active sequence number, the number of commit messages observed, and the quorum.

Beneath these values, the “Recent Events” log records every protocol action as it occurs. This includes message transmissions, replica transitions, election results, and commit decisions. The log is scrollable

and updates in real time, providing a traceable sequence of operations that users can consult for debugging, analysis, or instructional purposes. Its collapsible design enables users to hide or reveal the event history depending on their focus.

A key part of the side panel is the live “pBFT Explanation” window. This section gives a plain-language summary of whatever stage the protocol is currently in—whether that’s Client Request, Pre-Prepare, Prepare, Commit, or Reply—and updates automatically as new events occur. To make stage changes easier to spot, the explanation window briefly highlights itself whenever the system moves to a new phase. This small visual cue helps users follow the flow of the protocol and makes it easier to connect what’s happening in the canvas to the underlying pBFT logic.

To make the animation easier to follow, the side panel also includes a color-coded legend that explains the visual conventions used throughout the simulation. Each type of message is paired with its own color—green for Commit, orange for Prepare, blue for Pre-Prepare, yellow for Reply, and gray for idle or no activity. Whenever a new message appears on the canvas, the matching entry in the legend briefly flashes. This small interaction helps users connect the abstract protocol events to what they see on the screen, making the canvas much easier to interpret.

The last major section of the side panel is a built-in pBFT tutorial. This panel is always accessible, whether the simulation is running or not, and provides a clear, high-level explanation of the protocol’s purpose and structure. Its constant availability supports self-guided learning and gives users—especially those new to Byzantine fault tolerance—a place to anchor their understanding. By integrating this material directly into the interface, users can read about the protocol’s key ideas while simultaneously watching how those ideas play out in the simulation.

Most sections of the side panel use the same collapsible design, letting users open or hide panels as needed. This keeps the interface clean without taking away access to important information. Together, the system metrics, event log, stage explanations, color legend, and built-in tutorial turn the side panel into a helpful companion to the visual simulation. It supports both learning and debugging by giving users a clear view of what the protocol is doing and why.

## **5.5 Hosting Frontend on the Webserver**

# **6. Testing and Evaluation**

## **6.1 Functional Validat**

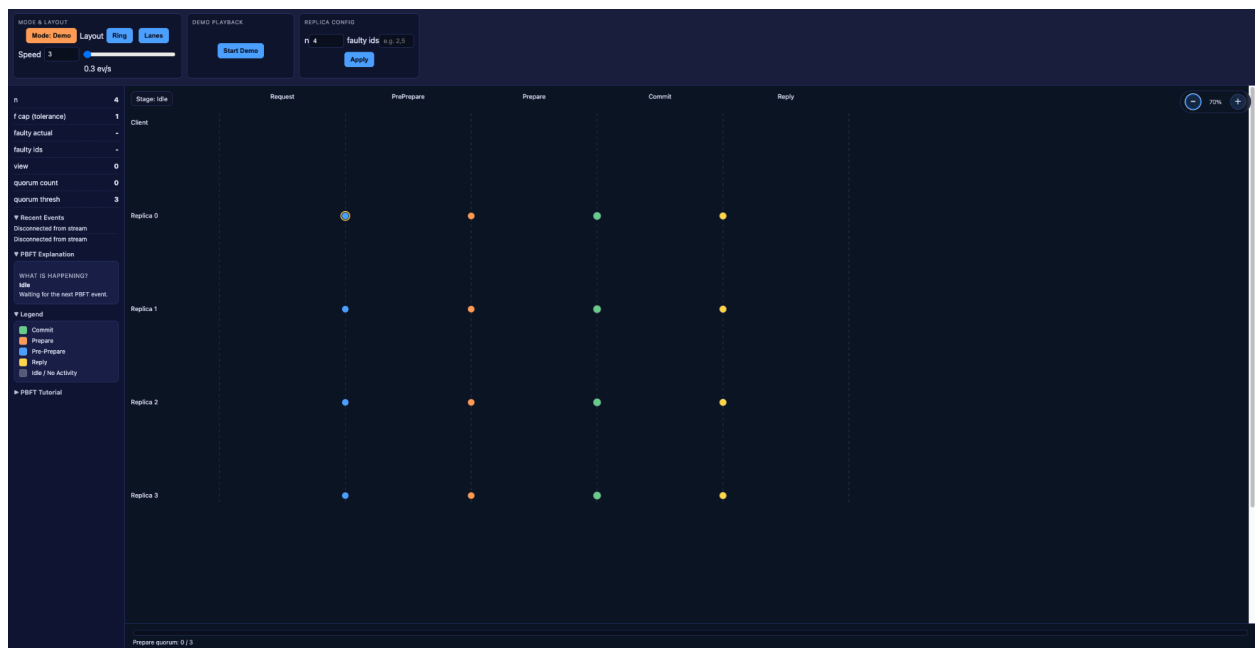
We validate both demo and live modes across baseline, complex, and edge scenarios to ensure the reducer, playback engine, and controls behave consistently regardless of event source. Each test case

exercises the same UI flows—layouts, speed, pause/step, replica controls, reset—to demonstrate parity between the offline demo driver and the live SSE stream.

### 6.1.1 Baseline Case

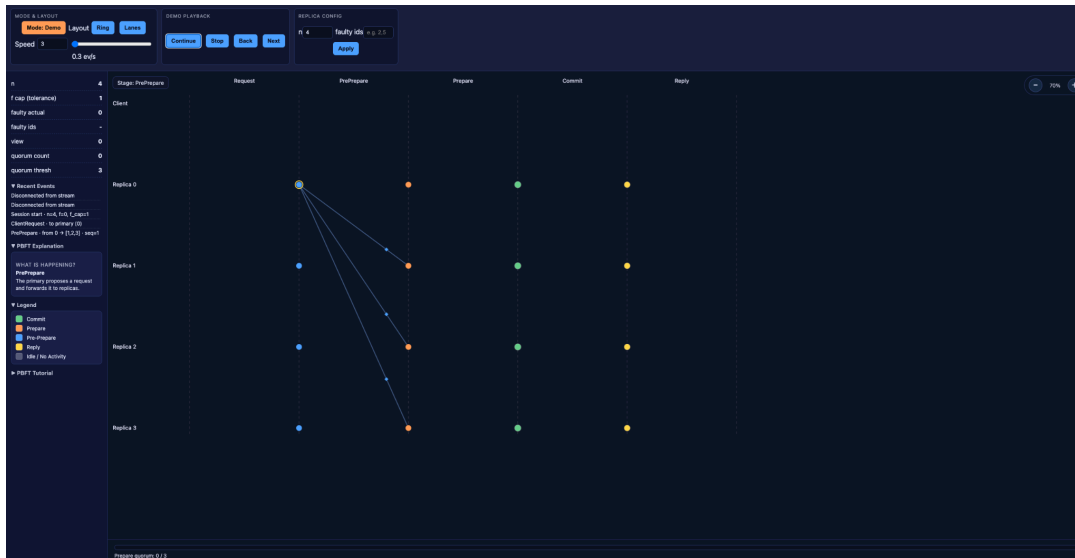
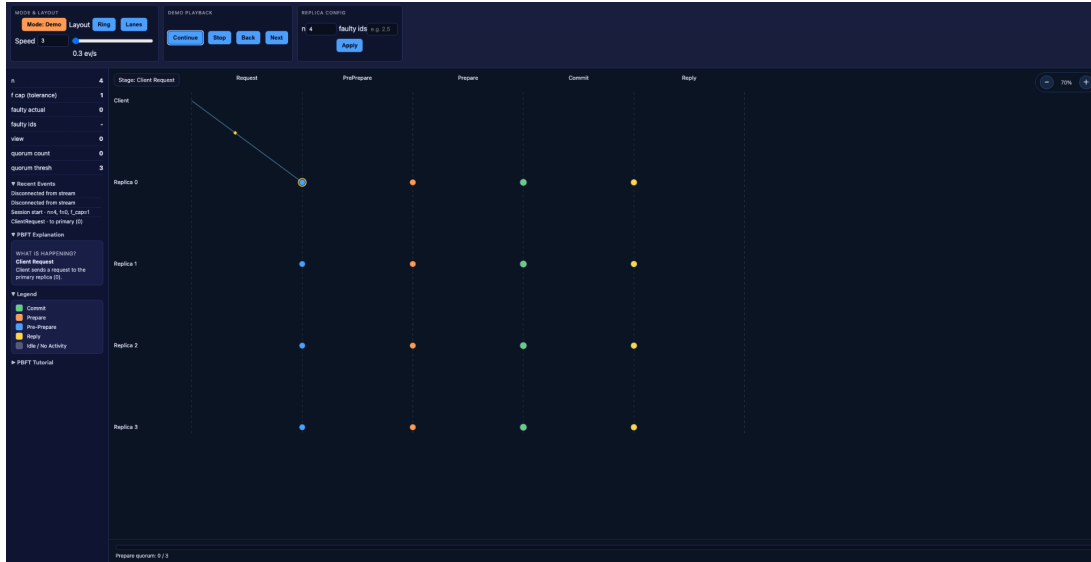
For the baseline validation we run both demo and live modes with default parameters (4 replicas, no faults, speed slider at 3) and exercise the key controls: pause/resume, speed, next/back, and layout toggles. The goal is to verify that the interface renders the expected PBFT stages and that the controls behave consistently across both modes.

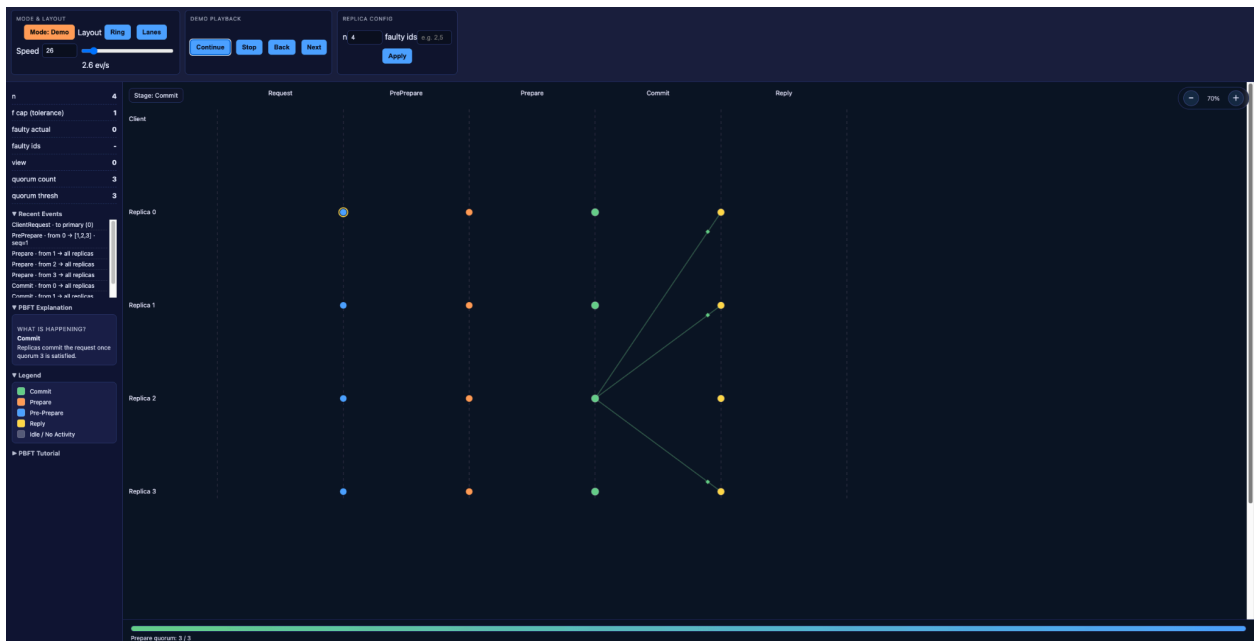
#### 6.1.1.1 Demo Mode

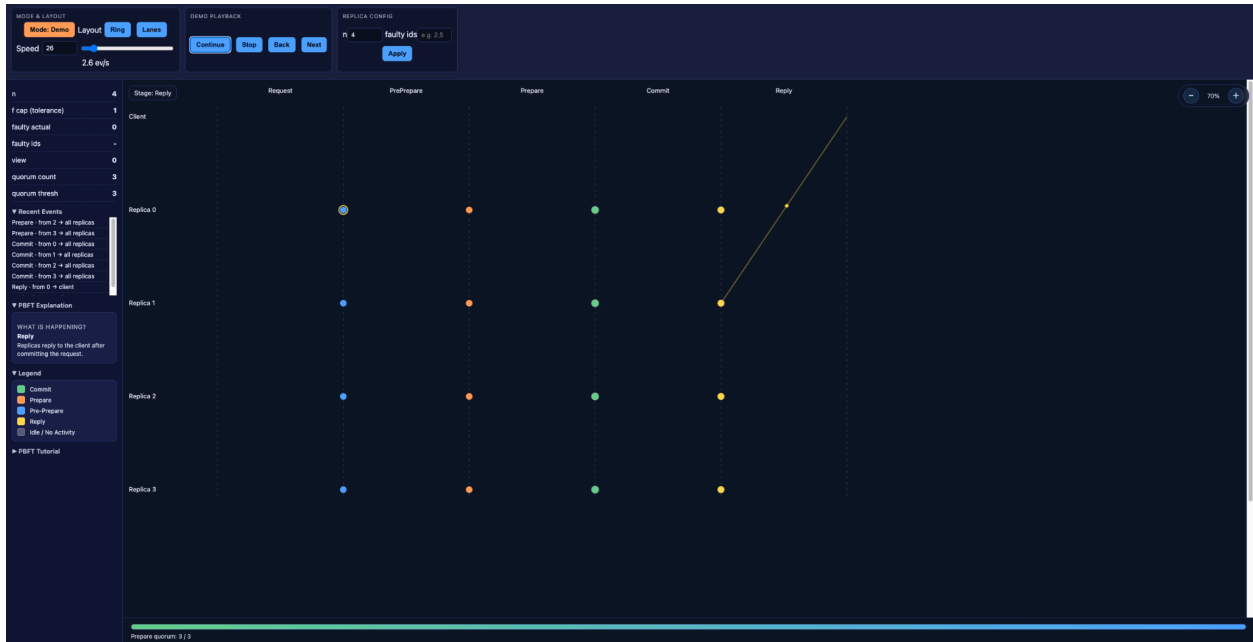


In demo mode the Top Bar exposes mode/layout toggles, playback controls, and replica configuration. The Canvas Panel defaults to the lanes layout, with the Sidebar showing quorum progress and event log entries.











## 4. Results and Conclusion

### 4.1 Results

This pBFT visualization tool can collect data from the backend C++ program and send it to the Redpanda cluster in the form of JSON logs that can be read and stored. These JSON logs can also be generated by user input to the visualization tool directly. The Python API, specifically Pandaproxy interface, is used to

transmit the JSON logs collected from the backend to the visualization tool. For the midterm presentation, this tool showed the collection and transmission of these logs and for the final portion, the system is able to stream the events correctly in real-time. The system can display the pBFT visualization that can be easy to understand for students and researchers. It shows the transactions between client and replicas (with at least 10 replicas in total). The frontend now delivers the full pBFT visualization experience in isolation: a reducer-driven state core, dual layouts (ring for communication patterns, lanes for stage-by-stage timelines), fault-aware demo mode, adjustable pacing, and responsive controls/telemetry. Users can step through or autoplay the protocol, watch quorum progress, inspect n/f/seq values, and toggle layouts without reloading—meeting our goal of an interactive, educational UI.

## 4.2 Difficulties

Initially, there were difficulties creating a standard plan for the design and implementation but by using GitHub version control and branches, it was possible to test multiple features and design scenarios before finalizing this frontend and backend setup. However, there were multiple issues faced with frontend rendering, API connectivity, and backend configurations. These are listed below:

### 4.3.1 Frontend

- **Layout schema tradeoffs:** Choosing between circular and lane representations required multiple prototypes to balance readability vs. dynamism. Supporting both meant doubling layout math and ensuring color/animation semantics stayed consistent.
- **Canvas responsiveness:** Making the renderer legible across replica counts demanded nuanced geometry (radius scaling, DPR handling, scrollable lanes) and careful performance tuning so pulses stay smooth.
- **State synchronization:** Demo, manual stepping, and socket-based data all share the same reducer pipeline; keeping them synchronized (especially when pausing/resuming or flagging faulty replicas) exposed timing edge cases that forced a refactor into modular hooks/components.
- **Redundant Kafka Consumption:** Every frontend client creates its own Kafka consumer, wasting resources and preventing shared offsets. A single shared consumer with a fan-out or caching layer is needed.
- **No Interactive Control Path:** The backend streams pBFT logs via SSE but provides no API for the frontend to issue commands (e.g., trigger a request, reset, replay). This prevents interactive testing and full simulator functionality.
- **No Replay / Pause / Stepping Support:** The system handles only live logs without buffering or indexing, making it impossible to pause, step through events, or replay an execution—critical for debugging and teaching use cases.

### 4.3.2 Backend

- **Incorrect Replica Configuration Causing Protocol Stalling:** We observed that the system consistently produced only the first three pBFT phases (request, preprepare, and prepare), but never logged any commit or inform messages regardless of configuration or logging verbosity. The root cause was a mismatch between the pBFT configuration files and the actual number of replicas deployed:
  - servers.data configured 10 replicas.
  - Our deployment script only launched 4 replicas.
  - The pBFT implementation (pbft\_demo\_json) does not support dynamically overriding replica count
  - As a result, the system believed it was running a 10-replica pBFT, but only 4 replicas existed
  - The protocol could never satisfy quorum conditions for prepare to commit so it stalled permanently.
- **Non-Monotonic and Inconsistent Timestamps Across Logs:** When analyzing logs gathered from multiple replicas, timestamps appeared out of order, sometimes jumping backwards or forwards by large margins. Events with later Kafka offsets sometimes had earlier timestamps.
  - **Suspected cause:** Each replica logs timestamps using its own unsynced system clock, while the Redpanda broker timestamps reflect order. Neither of these reflects the generation order (hardware limit). Therefore, the slightest discrepancy in the hardware clocks of each replicas/clients could result in issues.
  - **Impact:** Raw timestamps could not be used for reconstructing event timelines and ordering pBFT phases. However, since the pBFT phases already have a set order of execution, we can define the orders before the runtime, and not refer to the timestamps. In the case of the multi-shot version visualization, the timestamps and broker ordering can be used to track the order of requests (rather than phases). This works because the delay between each request would be large enough to mitigate the slight discrepancy of timestamps (at a micro scale), and they will be distinguishable.
- **Unexpected Message Counts (Missing Some commit Event Messages):** The classical pBFT normally produces predictable message counts (e.g., for 4 replicas and 1 client: 1 request, 3 preprepare,  $3 \times 9 = 9$  prepare,  $3 \times 4 = 12$  commit, 2-4 inform/reply). However, Redpanda logs showed only 4 commit messages. This has been consistent throughout each test, which leads to confusion about whether the protocol executed correctly.
- **Ambiguous Sender/Receiver Information in Logs:** There is a difficulty interpreting each log message. The log format does not provide explicit sender/receiver id and as a result, there's an ambiguity in deriving message direction from the log entirely.

### 4.3.3 Other

- One challenge we ran into was the inconsistent experience of connecting to GitHub and the web server over SSH across different machines. Team members using macOS, Linux, and various setups all had slightly different hurdles to overcome. macOS in particular tended to be stricter—it often required extra permissions, more `chmod` adjustments, and additional security prompts before allowing SSH connections or Git operations. Tasks that worked smoothly on Linux sometimes took more steps on a Mac. We also ran into Git issues related to SSH key management and machine-specific credentials, which occasionally blocked pushes and pulls until someone

re-authenticated or fixed their configuration. These differences slowed down development at times and showed how working across different operating systems can complicate collaboration on shared infrastructure.

## 4.3 Future Design

In an ideal scenario, it is expected that at least five or more nodes may be used simultaneously in the interactive visualization tool. The tool is expected to work in real-time as well as using a time dilation technique where previous and current states are visible using controls on the web page. In general, the tool is expected to handle delays, faults, and transmission of data (client and replica logs) from the backend to frontend.

For the frontend design, certain details should be expected in the future version as well:

- Replace pulse fades with explicit message markers (animated dots traveling between nodes) and tooltips/cursor selection so users can inspect individual messages in sequence.
- Add a log panel that records message history with timestamps and metadata, letting users scrub or filter events.
- Surface per-node status cards (e.g., current phase, outstanding messages, whether it's faulty/primary) alongside the canvas for quick inspection.
- Integrate richer filters (view/seq ranges, message types) and presets so instructors can tailor demonstrations for specific scenarios.

## 4.4 Conclusion

Overall, this tool can perform a visual simulation of multiple nodes and states based on the inputs provided to the frontend program. The goal is to reduce the logging overhead and display the actions and conditions of nodes in an interactive and educational scenario. It is assumed that users and programs will be able to access and modify this tool to be able to visualize custom scenarios in order to analyze the behaviour of the nodes in faulty or consensus-based scenarios.

In the frontend design, two views were implemented namely ring and lanes. Here the number of replicas, events, and messages are displayed using nodes. User interface features such as start, stop, and replay are used to control the processing of events. In this part, the timestamps and API connectivity to receive the events are important. It is possible to also write to the backend using user inputs stating the replica and message to produce another visualization.

In the backend design, the system programs are hosted on the webserver and the webpage is discoverable from the internet. The data is transmitted from the existing backend program to the wandlr and Redpanda setup where it can be organized to later transmit to the API connectivity module. The option to read and write (user input) is available and can be used to provide data to the backend from the frontend as well as to simply process a stream of events from the backend.

The system was tested at each phase and multiple design ideas were tested before implementing a chosen module or method. The frontend program can process the events in the form of individual parameters and can be used to create logs and messages to be displayed for the user. This tool can show one complete round of the pBFT demonstration. It was evaluated against a criteria of correctness, user friendly user interface, real-time results, and tracking or logging capabilities. The traces, resource/event tracking, and replay features of the tool work correctly and smoothly.

This project was an interesting task but a challenging one because of the system design and connectivity issues. In the future, multiple view changes or advanced user interface designs as well as simpler backend data modelling structures may be useful for students and researchers. However, in the final demonstration, the system was able to display events from the backend in a sophisticated user interface design displaying the nodes and events in a pBFT visualization. It shows that a pBFT visualization tool is useful for working with large numbers of replicas and may help with identifying the faulty conditions easily. Further research and testing will surely improve the current design and allow the system to handle complex scenarios and multiple view changes.

## 5. Task Breakdown

| Name                                                   | Tasks For Midterm                                                                                                                                                                                                                                                                                                       | Tasks For Final                                                                                                                                                                                                                                                                    |
|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Apurvata Kumar</i><br><br>Functionality and Testing | Setup ReactJS environment for testing to ensure that the client and replicas can be displayed. Create sample JSON logs to test connectivity and transmission of data. Assist with frontend and backend design tasks for data modification and connectivity. Work on presentation and report, including system diagrams. | Work with timestamps and design for multiple nodes and view changes according to the JSON data. Design the faults and consensus scenarios with the functionality of the frontend. Assist with other API connectivity, frontend, or backend tasks. Work on presentation and report. |
| <i>Daehan Lee</i><br><br>Data Ingestion Pipeline       | Architecture and backend infrastructure design. System orchestration via modified wandlr tool and bash script. Network flow management. Redpanda (proxy, broker, console UI) deployment via dockerized container. Streaming API outline & python venv setup. Report & presentation.                                     | Add multi-shot functionality. Improve performance and error handling for stressed setups (much more replicas & clients at faster request rates). Improve the overall design to allow the pipeline to handle any additional UI functionalities. Work on presentation and report.    |



|                                                                    |                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><i>Valeria Ramirez Osorio</i></p> <p>Controls and Narrative</p> | <p>Modify user interface design to include the user friendly features of the tool. Add a side panel for messages and displaying features. Work on presentation and report.</p>                                                                                               | <p>Design and implement the user interface for control and narrative interactions, prioritizing window action, visual displays, and text explanations. Build and test functionality to ensure responsive updates and ensure the system state is correctly reflected. Work on presentation and report.</p>                                                                                           |
| <p><i>Xinlu Yan</i></p> <p>Export Texts and Testing</p>            | <p>Work on API and connectivity to collect and transmit the events and data. Modify UI to include smooth transition of events. Define parameters for collecting and displaying events. Work with JSON logs to provide connection to UI. Work on presentation and report.</p> | <p>Test custom scenarios with connectivity issues and faults that affect data logging and visualization. Design a section in the tool to show the real-time actions and states of the nodes. Work on presentation and report.</p>                                                                                                                                                                   |
| <p><i>Yangyang Zhang</i></p> <p>Canvas and Animations</p>          | <p>Create the base for the visualization tool to include the nodes and animations. Add the different phases of pBFT. Modify UI features to work seamlessly in both lane and ring mode with different numbers of replicas. Work on presentation and report.</p>               | <p>Design and implement the main canvas visualization, including node placement, message animations, and color transitions for the Request–Preprepare–Prepare–Commit–Inform cycle. Ensure smooth frame rendering and responsive performance. Work on implementing the frontend on the webserver and ensuring that it is publicly accessible from the internet. Work on presentation and report.</p> |

## 6. References

1. James R. Clavin and Sisi Duan. 2019. ByzGame, a Visualized and Understandable BFT Consensus. In Proceedings of the 20th International Middleware Conference Demos and Posters (Middleware '19). Association for Computing Machinery, New York, NY, USA, 21–22. <https://doi.org/10.1145/3366627.3368109>
2. cmayurin. (2021). *pBFT Visualization (project blog & repository)*. <https://cmayurin.github.io/Visualization-pBFT-Blog>

3. Exploratory Systems Lab. (2023, December 6). *ResView-Visualizing ResilientDB*.  
<https://blog.resilientdb.com/2023/12/06/ResView.html>